

EBU6304 Software Engineering Group Project

2025-26

Group number	7	
member 1	QM no: 231221663	Name: Chang Ge
member 2	QM no: 231220390	Name: Yuxiang Tu
member 3	QM no: 231220714	Name: Mandan Dong
member 4	QM no: 231220253	Name: Maiyuan Cao
member 5	QM no: 231222279	Name: Hao Qin
member 6	QM no: 231222899	Name: Zixuan Wang

Final short report

1. Design

The system was designed as a lightweight Java Servlet/JSP web application for teaching assistant recruitment, with a clear separation of concerns across presentation, business logic, and persistence layers. This layered architecture improves maintainability, simplifies testing, and supports incremental Agile development. The presentation layer consists of JSP pages and static resources in `src/main/webapp`, while servlets act as controllers that process HTTP requests and forward data to the appropriate views. Business rules are encapsulated in service classes, and data access is implemented through JSON-backed DAO classes. This structure reduces coupling between modules and allows each layer to evolve independently.

A modular design principle guided the implementation. The codebase is organised into packages for `model`, `dao`, `service`, `servlet`, `filter`, and `util`. Domain models such as `Applicant`, `Job`, `Application`, `CV`, and `Message` represent the core business entities. Supporting view models, such as job recommendation and workload summary objects, allow the application to present computed information without mixing it with persistence concerns. This separation makes the design easier to extend, particularly when adding features such as recommendation scoring, workload warnings, and application review filters.

Role-based access control is another major design decision. Applicants, module organisers, and administrators each have different capabilities, so authentication and authorisation are handled centrally by filters and role-checking utilities. This prevents unauthorised access to protected routes and keeps access control logic out of individual servlets. The design also follows the coursework constraint of avoiding databases and heavy frameworks, so all application data is stored in JSON files. This approach keeps deployment simple and makes the system suitable for local testing and demonstration.

The application supports a workflow-driven recruitment process. Applicants can manage their profiles, upload CVs, browse jobs, view skill gaps, and apply for positions. Organisers can create and manage job postings, review applicants, and update application status. Administrators can supervise jobs, configure policy settings, and export workload reports.

These features were chosen to provide an end-to-end recruitment prototype while keeping the scope manageable for iterative delivery.

From an Agile perspective, the design evolved gradually. Early iterations focused on core data structures and persistence, then on authentication and primary application flows, and later on recommendation, workload, and reporting features. This incremental approach reduced risk and allowed the team to validate each module before extending the system.

2. Implementation

Implementation followed an iterative build strategy aligned with Agile development. The project started with the core data model and file-based persistence layer, then expanded through successive iterations to cover user workflows, organiser review functions, and administrative reporting. Each iteration produced a working increment, which supported continuous integration and early feedback. This approach was especially useful for a coursework project where functionality needed to be demonstrated progressively rather than implemented all at once.

The project uses Maven as the build tool and produces a WAR artifact for deployment to a Servlet 6 compatible container such as Tomcat 10.1+. The build process is straightforward: developers run `mvn test` to execute the test suite and `mvn package` to generate the deployable application. Runtime data is stored in JSON files, which keeps the application independent of external databases and makes local setup reproducible. The repository also includes a deployment helper script, which simplifies deployment during development and demo preparation.

Build Plan

The final build environment uses Java 17, Maven 3.9 or later, and Apache Tomcat 10.1 or another Servlet 6 compatible container. Developers first run `mvn test` to execute the automated test suite, then run `mvn package` to create `target/ta-recruit.war`. For local demonstration, `run-local.ps1` builds and deploys the WAR to Tomcat, configures the runtime data folder, and can open the application in a browser. The runtime data location is controlled by `TARECRUIT_DATA_DIR` or the `-Dtarecruit.data.dir` JVM property, and the default login path is `/ta-recruit/login`. Demo accounts are provided for applicants, module organisers, and the administrator so acceptance testing can be repeated from a clean local setup.

GitHub collaboration was central to the implementation workflow. The repository shows a clear history of incremental commits that reflect feature-by-feature development. Feature branches such as `dev/cmy`, `dev/dmd`, `dev/gc`, `dev/qh`, `dev/tyx`, and `dev/wzx` were used to isolate individual development tasks, while the test branch acted as the integration branch for combining and validating work before release. For the final submission, the latest tested version has been merged into `main` so the released software can be inspected from the main branch, while the feature branches and commit history remain visible as evidence of individual contribution. This branch-based workflow allowed team members to work on

different parts of the system in parallel while keeping the final release branch stable for testing and demonstrations.

Version control was used not only to store code, but also to demonstrate team process. Commits were grouped around meaningful tasks such as data model creation, workflow integration, organiser UI synchronisation, admin dashboard development, skill matching, applicant recommendations, workload warnings, and CSV export. This pattern is consistent with Agile sprint delivery: small changes were integrated frequently, reviewed through testing, and then extended in the next iteration. The presence of documentation updates and testing commits at the end of the history also suggests a deliberate final stabilisation phase before submission.

Overall, the implementation strategy prioritised clarity, testability, and incremental delivery. Rather than relying on a complex framework stack, the team used simple Java web technologies and disciplined module boundaries to keep the system maintainable and aligned with the coursework requirements.

3. Testing

3.1 Testing Strategy

The testing strategy for the TA Recruitment System focuses on role-based functional verification, workflow correctness, and data persistence validation. The system supports three distinct roles: Applicant, Organiser, and Admin, each with specific operations. Testing was conducted to ensure that each role performs the correct actions, access control is enforced, and the workflow between applicants, organisers, and administrators is consistent and error-free.

Testing is divided into three layers:

Unit Testing – Each service and DAO class was tested individually using JUnit to verify business logic, validation rules, and data access behavior. Examples include ApplicantService, CvService, JobService, ApplicationService, and RecruitmentPolicyService.

Integration Testing – Key workflows combining multiple services were tested to ensure end-to-end correctness. For instance, an applicant creating a profile, uploading a CV, applying for a job, and the organiser reviewing that application.

System Testing – The deployed web application was tested in a local Tomcat server environment using demo accounts to simulate real user interactions. Functional testing, role-based access, and edge cases were covered.

The main goals were:

Verify that role-based access control prevents unauthorized operations.

Ensure that the recommendation and skill matching calculations are accurate.

Confirm that application submission rules, duplicate prevention, and withdrawal processes function correctly.

Validate that system metrics and dashboards accurately reflect the current state.

3.2 Techniques Used

Manual Functional Testing – Using the steps outlined in the video script, each operation was manually performed and observed. For example, logging in as an applicant and testing job applications with complete and incomplete profiles.

Boundary Testing – Testing limits of application capacity, skill match percentages, and slot availability to ensure the system handles edge cases gracefully.

Error Handling Verification – Intentionally triggering errors, such as attempting duplicate applications or exceeding limits, to confirm that validation messages appear as expected.

Unit Testing with JUnit – Automated tests covering DAO operations, service-level logic, and workflow scenarios.

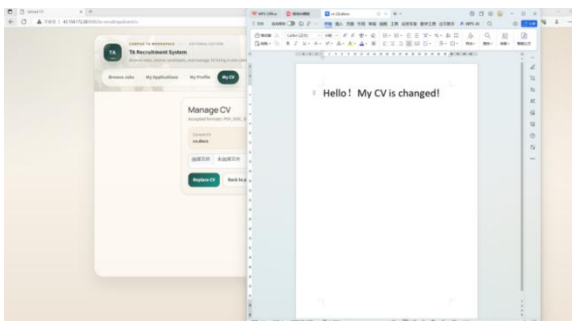


Figure 3.1: CV Editing Page.

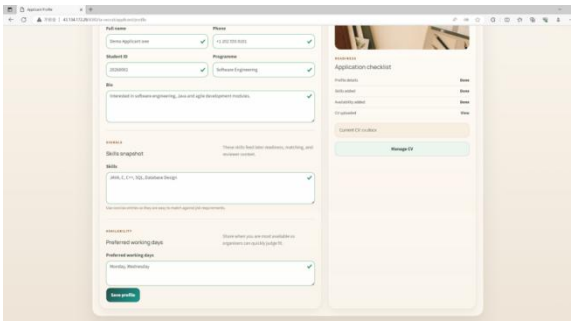


Figure 3.2: Edit Profile Page

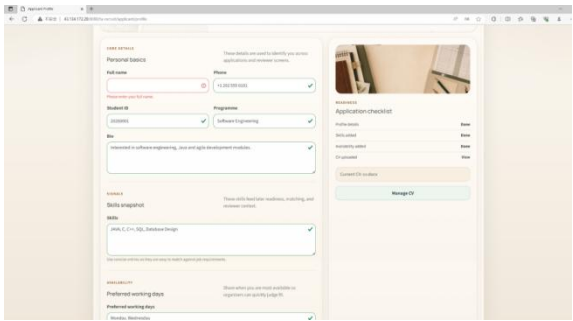


Figure 3.3: Edit Profile Page(Incomplete profile prevents submission)

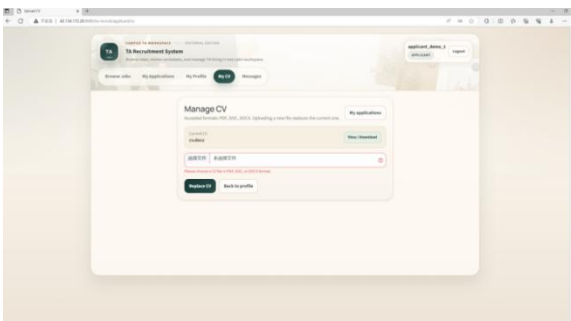


Figure 3.4: CV File Type Selection (submission is blocked if no CV is uploaded).

3.3 Test Case Design

Test cases were designed according to roles and core workflows.

3.3.1 Applicant Role

Test Case	Input	Expected Result	Actual Result
-----------	-------	-----------------	---------------

Profile Completion	Leave some fields empty	Submission blocked, validation message shown	Passed
CV Upload	Upload PDF, DOC, or DOCX file	CV successfully stored, status updated	Passed
Job Search	Search "Java"	Matching jobs displayed with skill match panel	Passed
Apply Job	Apply with full profile and CV	Application saved, status PENDING	Passed
Duplicate Apply	Apply again for same job	Application blocked, warning displayed	Passed
Application Limit	Apply when limit reached	Apply button disabled	Passed
Withdraw Application	Withdraw PENDING application	Status changes to WITHDRAWN	Passed

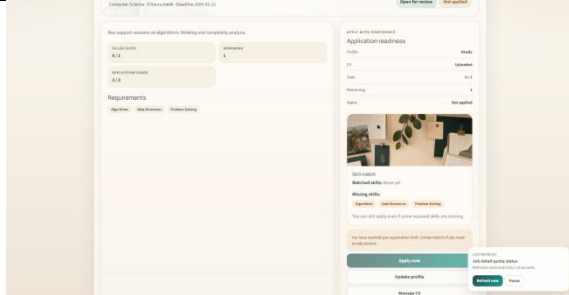


Figure 3.5: Applicant Job Selection Page (application requirements are not met).

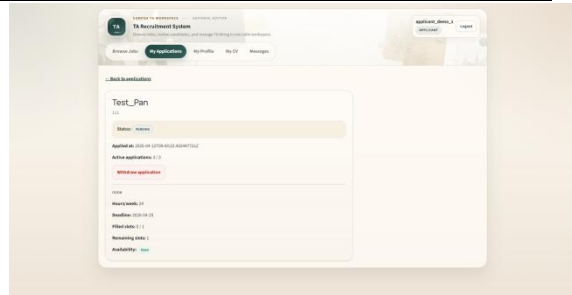


Figure 3.6: Applicant Application Details Page.

3.3.2 Organiser Role

Test Case	Input	Expected Result	Actual Result
View Job Postings	Open job list	Job titles, department, slots, hours, deadlines visible	Passed
Create Job	Fill new job form	Job created successfully (demo-safe)	Passed
Review Applications	Filter by status, match score	Correct applications displayed	Passed
Accept/Reject Application	Accept PENDING application	Status changes to ACCEPTED or REJECTED	Passed
CV Access	Open applicant CV	Only accessible for own job	Passed

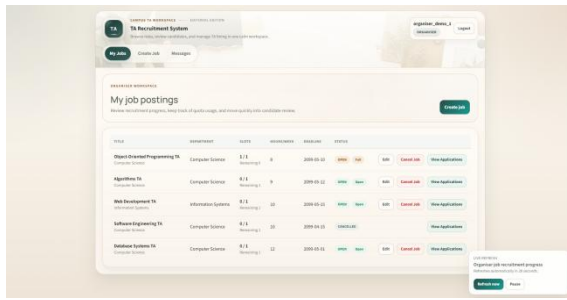


Figure 3.7: Organiser Job Postings Page.

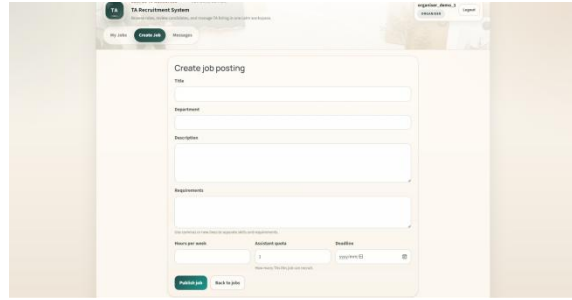


Figure 3.8: Organiser Create Job Page.

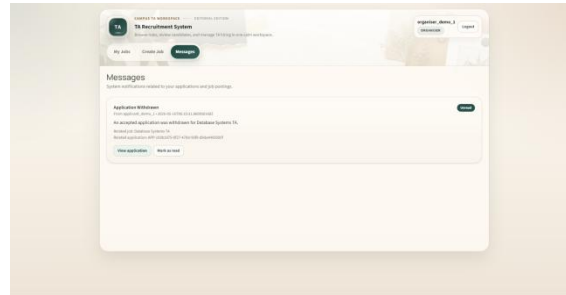


Figure 3.9: Organiser Messages Page.

3.3.3 Admin Role

Test Case	Input	Expected Result	Actual Result
Dashboard Metrics	Open dashboard	Users, jobs, applications, limits displayed	Passed
Global Settings	Adjust limits	Effective limits updated	Passed
Workload Monitoring	View workloads	Threshold status correct, CSV export works	Passed

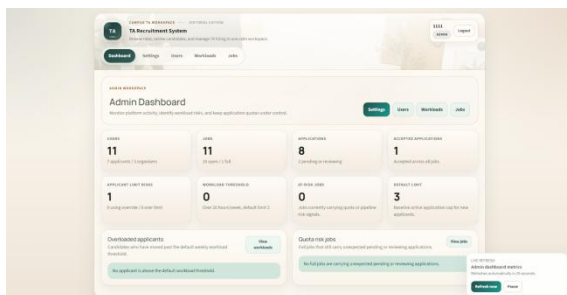


Figure 3.10: Admin Dashboard Page.

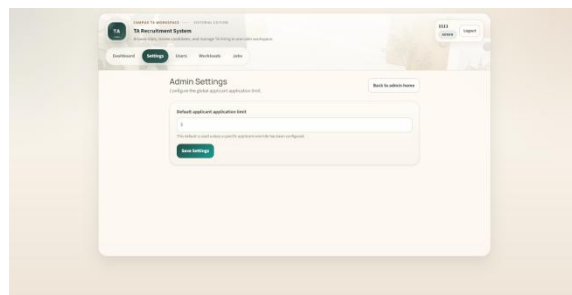


Figure 3.11: Admin Settings Page.

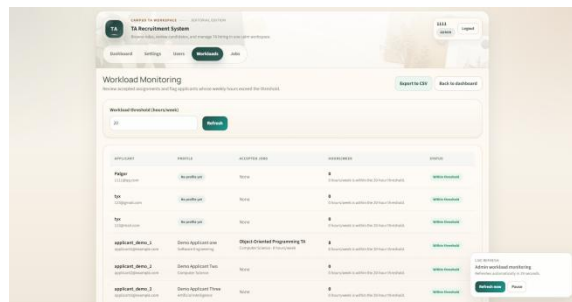
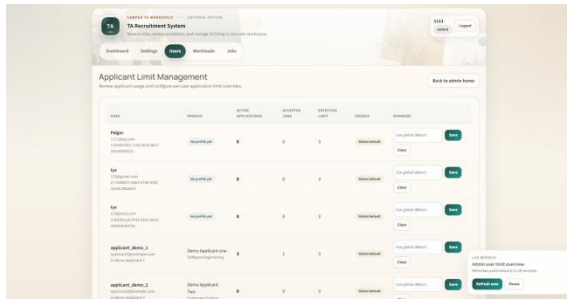


Figure 3.12: Admin Users Management Page. Figure 3.13: Admin Workloads Page.

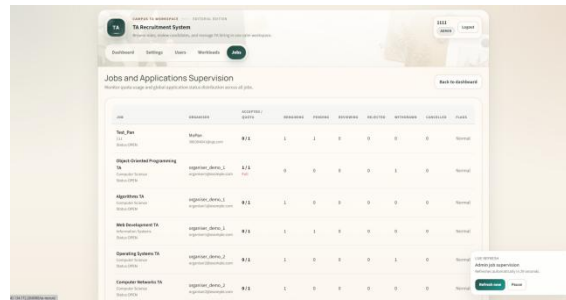


Figure 3.14: Admin Jobs Management Page.

3.4 Test Results

The overall testing outcome demonstrates that the system functions as intended:

Role-based Access Control – Verified via login tests and route visibility checks; all users can only access their permitted pages.

Workflow Accuracy – End-to-end workflow from profile creation to job assignment completed successfully.

Validation & Error Handling – All intended validation messages appear for incomplete profiles, missing CVs, duplicate applications, or exceeding limits.

Skill Matching & Recommendations – Skill comparisons and match scoring operate according to the rule-based calculation described in the video.

Data Persistence – JSON-backed DAOs store and retrieve data correctly, confirmed via local persistence checks and JUnit tests.

All automated and manual tests passed with no critical failures. The automated suite was verified with mvn test, passing 193 tests with 0 failures and 0 errors. The build was also verified with mvn package, which generated target/ta-recruit.war for Tomcat deployment. Code documentation was verified by generating JavaDocs with mvn javadoc:javadoc, producing the API documentation under target/site/apidocs. Minor interface improvements were noted but did not affect functionality.

3.5 Summary

Through this multi-level testing strategy, the system has demonstrated:

Correct enforcement of business rules and application policies.

Accurate role-specific functionality for applicants, organisers, and administrators.

Reliable error handling and user feedback.

Effective data persistence and workflow integrity.

This ensures the TA Recruitment System is robust, reliable, and ready for demonstration and deployment in a coursework context.

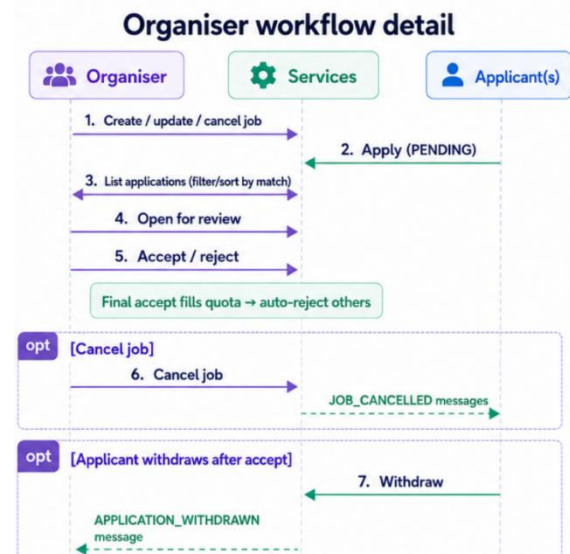
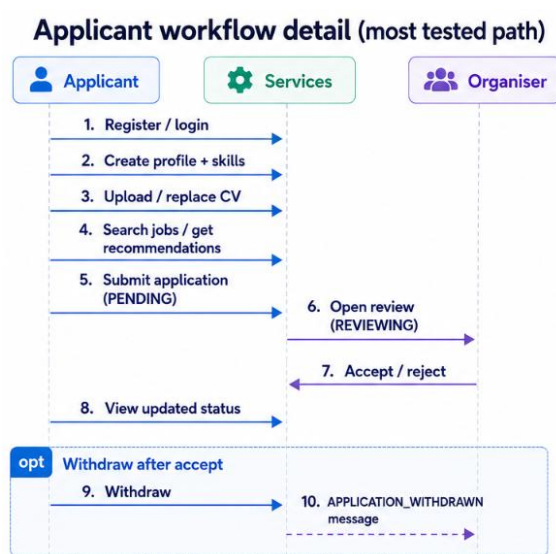


Figure 3.15: Applicant Workflow Detail (Most Tested Path). Figure 3.16: Organiser Workflow Detail.



Figure 3.17: Admin Workflow Detail.

4. The use of Generative AI (GenAI)

Generative AI was used as a development aid rather than as an authoritative source of implementation decisions. In this project, GenAI helped the team review the handout requirements, draft README and user manual text, suggest validation and workflow test

cases, and debug service-level logic during implementation. It was also useful when turning technical notes into clearer report and documentation drafts.

The main advantage of AI assistance was speed. It reduced time spent on repetitive drafting and helped the team quickly compare possible approaches for testing, documentation, and workflow validation. For example, AI-supported review helped identify final-assessment gaps such as JavaDocs, user manual screenshots, build instructions, and more precise testing evidence.

However, AI-generated output was never treated as automatically correct. Suggestions were checked against the actual codebase and the coursework constraints: the system had to remain a lightweight Servlet/JSP application, store data in JSON files, avoid database integration, and keep recommendation behaviour explainable. Any suggested design or code idea had to fit the existing DAO, service, servlet, and JSP structure before being accepted.

The product features that appear AI-like are implemented as deterministic and explainable services. SkillMatchService compares applicant skills with job requirements and reports matched and missing skills; JobRecommendationService ranks open jobs using the skill match score, deadline, and title; and AdminService calculates workload warnings from accepted assignments and weekly-hour thresholds. This rule-based design means the team can explain why each recommendation, skill gap, or workload warning appears during the viva.

5. Git history, Agile collaboration, sprint progress, and contribution patterns

The Git history shows a clear Agile-style progression. The earliest commits establish the data model and persistence foundation, followed by workflow integration and role-based UI synchronisation. Later commits add the admin dashboard, settings, and supervision pages. The middle of the history introduces richer business logic such as status transitions, withdrawal messaging, cancellation handling, and end-to-end message flow. The final phase adds skill matching, organiser match scores, applicant recommendations, workload warnings, CSV export, and iteration 4 regression tests.

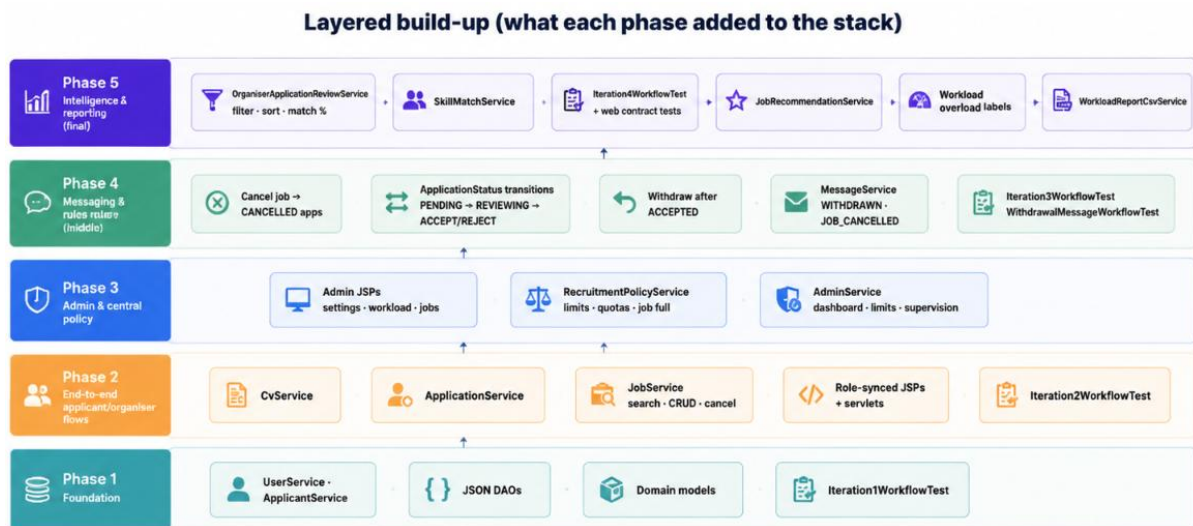


Figure 3.18: Layered Build-Up (What Each Phase Added to the Stack).

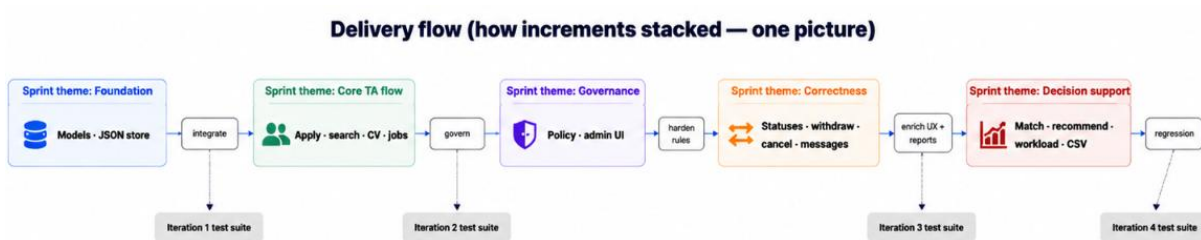


Figure 3.19: Delivery Flow (How Increments Stacked — One Picture).

This progression suggests sprint-based delivery rather than one-time implementation. The team appears to have moved from infrastructure to core workflow, then to advanced features and stabilisation. That is consistent with Agile collaboration: each iteration delivered a usable increment, and later work refined or extended earlier modules. The commit messages also indicate that different members contributed to different functional areas, such as front-end fixes, policy services, admin pages, messaging, and final testing.

Individual contribution and reflection

QM no: 231221663

Name: Chang Ge

Main contribution:

I was responsible for the applicant-side core of the TA recruitment system, covering application workflow, CV-related persistence support, and applicant application-limit policy. In the replay plan from de01788 to HEAD, my work began with applicant limit policy persistence (ApplicantLimitPolicy, DAO, and tests), which gave administrators a foundation for later limit controls. I then integrated ApplicationService and its tests so applicants could submit applications under business rules such as profile completeness, CV requirements, and duplicate-application checks. I synchronized applicant-facing servlets—including application detail, my applications, and registration-related routes—with the evolving workflow. I extended the applicant limit DAO for admin settings so global and per-applicant limits could be managed safely. I also contributed documentation updates (current status and task allocation) and Iteration 4 application workflow test coverage. Together, these deliverables connected applicant profiles, CVs, and applications into a coherent backend path that organisers and administrators could rely on.

Reflective statement:

This project taught me how to own a vertical slice of backend logic while staying aligned with teammates who owned jobs, policy, and UI. The main challenge was keeping application state consistent when other members added recruitment policy, admin limits, and organiser review features; I often had to re-read integration points in ApplicationService and DAO layers before committing. Splitting persistence from business rules—JSON DAOs below services—made the design clearer, but it also required careful ordering of replay commits so each step still compiled and passed tests. I improved at writing JUnit tests for service behaviour and at coordinating through shared models rather than duplicating rules in servlets. Overall, working in a six-person, commit-ordered replay schedule strengthened my skills in incremental delivery, interface stability, and communicating ownership through documentation and tests.

QM no: 231220390

Name: Yuxiang Tu

Main contribution:

I owned job-related backend capabilities: persistence helpers, browsing and search, and later skill matching and job recommendations. My first commits extended job persistence, file storage utilities, and JobDaoTest, establishing how jobs are stored and loaded from JSON. I aligned JobServiceTest with the integrated application workflow so job availability, quotas, and application rules stayed consistent. I delivered job browsing servlets (JobListServlet, JobDetailServlet) so applicants could search and view open roles. After UI synchronization, I fixed JobService to support admin supervision and organiser edit flows without breaking existing behaviour. In the final phase I implemented SkillMatchService and JobRecommendationService (with tests), enabling match scores for organisers and ranked recommendations for applicants. My modules sit between raw job data and higher-level recruitment decisions, so other members could build review screens and applicant job lists on a stable job layer.

Reflective statement:

I learned that job logic is a hub: almost every feature—applications, admin supervision, recommendations—depends on correct job state and search behaviour. The hardest part was evolving JobService without regressions when policy, applications, and admin views changed in parallel; regression tests and replay commit order were essential. Implementing skill matching forced me to think about domain rules explicitly (matched vs missing skills, scoring) instead of hiding logic in JSPs. The replay assignment also trained me to make small, reviewable commits tied to real source history rather than one large dump. I developed stronger skills in service design, test-driven alignment with workflow tests, and reading others' changes to avoid breaking servlet contracts.

QM no: 231220714

Name: Mandan Dong

Main contribution:

I led system-wide configuration, recruitment policy, admin backend services, and global error handling. I introduced SystemSettings persistence and SettingsService, which later supported applicant application limits and admin configuration. I implemented RecruitmentPolicyService and its test suite so limits, job quotas, “job full” rules, and related checks live in one place instead of scattered across servlets. For admin functionality I built and extended AdminService, admin view models, and servlets for settings, users, home, and dashboard supervision (workload and job oversight). I added the global ErrorHandlingFilter so unexpected failures are logged server-side and users see safe error pages. In later iterations I completed admin workload warning logic and CSV export on the service side. I also organized backlog and coursework documentation during the docs reorganize phase. My work aimed to keep governance and policy explicit, testable, and separate from presentation.

Reflective statement:

Centralizing policy in RecruitmentPolicyService was the most valuable design lesson: once rules were clear, other members could add features without re-implementing limits in every servlet. The challenge was timing—admin and applicant modules needed stable policy APIs early, so I had to deliver settings and policy before many UI features were finished. Error handling reminded me that security and UX both matter: users should not see stack traces, but developers still need logs. Coordinating with gc on limits and with wx on admin pages required clear contracts and tests. Through this process I improved at designing shared services, writing policy-focused unit tests, and documenting phases so the group could track progress across iterations.

QM no: 231220253

Name: Maiyuan Cao

Main contribution:

I focused on the module organiser path: servlets for reviewing applications, admin supervision routes on the organiser side, and Iteration 4 review enhancements. I synchronized organiser application servlets (job applications list, application detail, organiser jobs) during the UI alignment phase so organisers could see and act on applications for their own jobs only. I added admin job supervision and workload servlet routes that connect admin views to backend data. I completed the organiser job create/edit flow fixes so posting and updating jobs worked reliably with validation and ownership checks. For Iteration 4 I delivered organiser applicant match scores and optimized application list filtering/sorting via OrganiserApplicationReviewService, helping organisers compare candidates by status, keyword, and match percentage. My contributions emphasized resource ownership—

organisers must not access other organisers' jobs—and a clear review chain from listing applications to viewing details and recording decisions.

Reflective statement:

Organiser features sit between applicant submissions and admin oversight, so I constantly balanced permissions with usability. The toughest challenges were owner validation (every servlet had to check job ownership) and keeping status transitions consistent with ApplicationService rules defined elsewhere. When match scores and filters were added, I learned to expose read models (review views) instead of pushing complex logic into JSPs. Working through ordered replay commits taught me to integrate incrementally and to test organiser flows after policy or job changes landed. I grew more confident with servlet design, authorization boundaries, and collaborating with tyx on JobService and with qh/wzx on matching UI behaviour.

QM no: 231222279

Name: Hao Qin

Main contribution:

I owned the applicant-facing presentation layer: JSP workflow pages, styling, form feedback, and Iteration 4 applicant UX for recommendations and skill gaps. I synchronized core applicant pages—job list, job detail, applications list, and application detail—with backend attributes so users could browse roles, apply, and track status clearly. I improved real-time page feedback and shared CSS for applicant views during the experience enhancement phase. I implemented Bootstrap-style validation feedback on applicant and authentication forms, reducing silent failures and aligning with Iteration 3 quality goals. For Iteration 4 I updated job detail to show matched and missing skills (skill gap) and the job list to display recommended jobs, connecting tyx's backend recommendation logic to a transparent applicant UI. My goal was to make the applicant journey understandable: prepare profile and CV, discover suitable jobs, apply, and follow outcomes without exposing internal implementation details.

Reflective statement:

Frontend work in a servlet/JSP project meant I depended heavily on stable request attributes and clear contracts from backend teammates. The main difficulty was staying in sync when servlets, status enums, and policy rules changed—pages broke if attribute names or status labels drifted. Form validation feedback taught me to think from the user's perspective first, then map messages to server-side rules. Collaborating with wxz on shared navigation, styles, and contracts helped me see how “web contract” tests protect JSP assumptions. I developed skills in layout consistency, status display, and iterative UI polish across iterations rather than leaving all UX to the end.

QM no: 231222899

Name: Zixuan Wang

Main contribution:

I carried admin and organiser frontends, cross-role workflow regression tests, phase documentation, and several Iteration 3–4 integration commits. Early replay commits recorded phase completion reports for persistence and policy milestones. I added workflow regression tests (Iteration2WorkflowTest, Iteration3WorkflowTest, WithdrawalMessageWorkflowTest) and later Iteration 4 documentation. I synchronized organiser JSP workflow pages and built admin settings, users, dashboard, jobs, and workloads pages plus shared navigation. I improved admin/organiser real-time refresh behaviour and organized reports and planning documents. I contributed form validation contracts for admin and organiser forms, iteration completion notes, organiser filter UI contracts, admin workload warning pages, and workload CSV export wiring. On the final replay commit I added the Iteration 4 completion report. Overall I acted as integrator: connecting backend features to usable pages and guarding end-to-end behaviour with workflow tests.

Reflective statement:

Because I touched many areas, the biggest challenge was context switching between admin governance UI, organiser review UI, tests, and docs—without becoming a bottleneck. Ordered replay commits forced discipline: each step had to leave the repo buildable for the next person. Workflow tests were my safety net; they taught me to think in user journeys (apply → review → accept → workload warning) rather than isolated files. Documentation work improved my ability to summarize what the system actually shipped versus what was planned. I strengthened skills in integration testing, JSP/servlet alignment, and team coordination—especially when pairing with dmd on admin/error handling and with cmy/qh on organiser/applicant screens.